

Assignment 4

WiDS

Jan 2026

Problem 1

You are building a neural network to recognize handwritten digits $(0, 1, \dots, 9)$. Consider two different architectural approaches for the output layer:

- **Approach A:** You use a single output neuron with a Linear activation function. You train the network to output the numerical value of the digit (e.g., if the image is a "2", the target label is $y = 2.0$).
- **Approach B:** You use 10 output neurons with a Softmax activation function. You train the network using One-Hot encoded targets (e.g., if the image is a "2", the target is $[0, 0, 1, 0, \dots]$).

Question: Why is **Approach A** generally considered a bad idea for this specific problem?

Problem 2

Consider a neural network with two hidden layers: $p = 4$ input units, 2 units in the first hidden layer, 3 units in the second hidden layer, and a single output.

- Draw a picture of the network, similar to standard architectural diagrams (showing nodes and edges).
- Write out an expression for $f(X)$, assuming ReLU activation functions ($\sigma(z) = \max(0, z)$). Be as explicit as you can!
- Now plug in some values for the coefficients (weights and biases) and write out the calculated value of $f(X)$.
- How many parameters (weights + biases) are there in total?

Problem 3

A single neuron uses the ReLU activation function: $f(z) = \max(0, z)$, where $z = \sum w_i x_i + b$.

- Plot or describe the derivative $f'(z)$ with respect to the pre-activation input z .
- During training, a specific neuron in a hidden layer ends up with weights such that for *every* example in the training set, the pre-activation sum z is negative ($z < 0$). Write down the value of the gradient of the loss with respect to the weights w_i for this specific neuron.
- Based on your answer in (b), will the weights of this neuron ever change during subsequent steps of Gradient Descent? Explain why or why not.

Q.1

Approach A is a bad idea because it treats digit recognition as a regression problem instead of treating it accurately as a classification problem. The problem in such a case is the false belief of the neural network that there is a mathematical relationship between guesses.

eg. It assumes that predicting a '3' when the answer is '2' is only slightly wrong (as it's mathematically close), meanwhile predicting an '8' instead of '2' is much worse, but actually there's no relevance to this as any wrong guess is simply WRONG and there's no error room that's more acceptable than another.

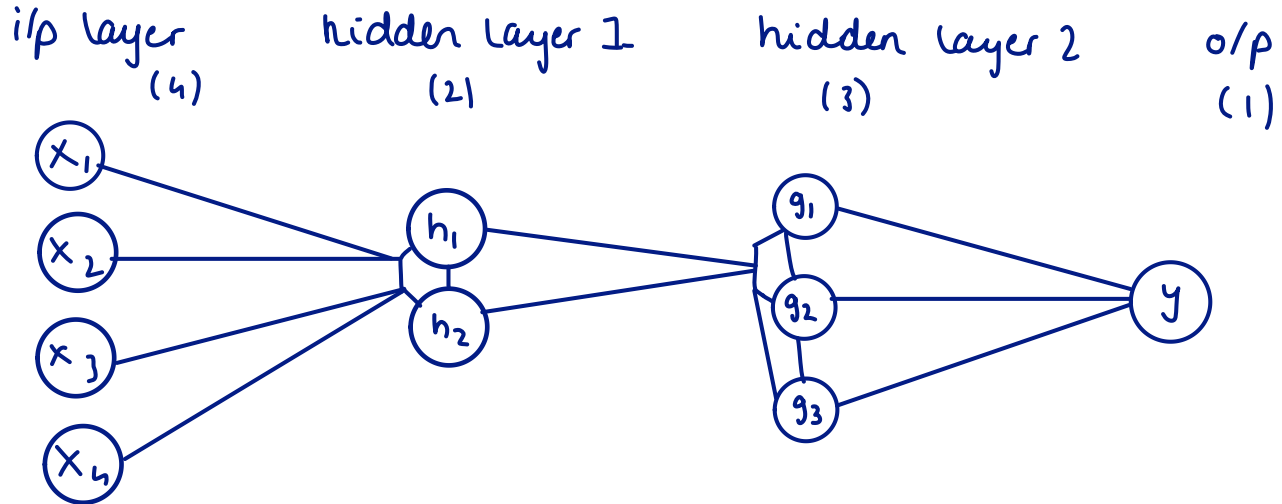
Another issue is ambiguity in predictions, as linear outputs of decimals i.e. 2.7 or 4.2, which are not 'digits' per se, are also possible then, which makes no sense.

∴ Choosing approach B is more logical, as it has probabilities for each digit and also penalises incorrect guesses reasonably.

→ Very nice 3B1B video that dives deeper, which I found very helpful & enlightening. : **What is a neural network:**

https://www.youtube.com/watch?v=aireAruvnKk&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&index=1

Q.2
a.y



b.y $\sigma(z) = \max(0, z)$ we take $X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$

for the first hidden layer: (4 i/p & a bias)

$$h_1 = \sigma(w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + w_{14}x_4 + b_1)$$

$$h_2 = \sigma(w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + w_{24}x_4 + b_2)$$

$\therefore$ for the second layer:

$$g_1 = \sigma(v_{11}h_1 + v_{12}h_2 + c_1)$$

$$g_2 = \sigma(v_{21}h_1 + v_{22}h_2 + c_2)$$

$$g_3 = \sigma(v_{31}h_1 + v_{32}h_2 + c_3)$$

$\therefore$ for the final o/p layer:

$$f(x) = u_1g_1 + u_2g_2 + u_3g_3 + d$$

(o/p from n-1 layer are taken as i/p for nth layer)

C.4 Taking arbitrary values: $x = [1 \ 2 \ -1 \ 0]$

① first hidden layer: $w_1 = [1, 0, 1, 0]$, $b_1 = 0$

$\therefore h_1 = 0, h_2 = 1$ $w_2 = [0, 1, 0, 1]$, $b_2 = -1$

② second hidden layer: $v_1 = [1, 1]$, $c_1 = 0$

$\therefore (g_1, g_2, g_3) = (1, 1, 0)$ $v_2 = [-1, 1]$, $c_2 = 0$

$v_3 = [0, 1]$, $c_3 = 1$

③ o/p layer $u = [1, 2, -1]$ $d = 0$

$\therefore f(x) = 1 \cdot 1 + 2 \cdot 1 - 1 \cdot 0 = 3$

$\therefore f(x) = 3$

d.) Total Parameters (counting weights & biases)

1/p $\rightarrow$ 1st hidden layer: $4 \text{ i/p's} \times 2 \text{ neurons} = 8 \text{ weights}$

biases: 2 (one for each neuron)

1st $\rightarrow$ 2nd hidden layer: $(2 \times 3) \text{ neurons} = 6 \text{ weights}$

biases: 3

2nd $\rightarrow$ O/p layer: $(3 \times 1) \text{ neurons} = 3 \text{ weights}$

biases: 1

$\therefore$ weights: $8 + 6 + 3 = 17$ biases: 6

$\therefore$ total parameters: $17 + 6$
 $= 23$

Q.3

a.) Given: $f(z) = \max(0, z)$

→ ReLU function has diff. behaviour on either side of $z = 0$:

$$f'(z) = \begin{cases} 0 & : z < 0 \\ \text{undefined} & : z = 0 \\ 1 & : z > 0 \end{cases}$$

∴ for $z < 0$:

The o/p is 0 & small changes don't change o/p.

for $z > 0$:

function is linear with slope = 1

b.) Given: for every training eq., the preactivation is -ve: $z < 0$

as $f'(z) = 0$ for $z < 0$

∴ neuron is inactive (o/p is always 0)

∴ ReLU o/p & gradient is 0.

c.) No, the weights will not change.

As the gradient is zero for every training eq., the update step is always 0, resulting in no change in weights.

This is also called a 'dead ReLU' as learning completely stops for this neuron.

Problem 4

We often add an L2 penalty term to our loss function to prevent overfitting. Let the total loss function be:

$$J_{total}(w) = J_{data}(w) + \frac{\lambda}{2}w^2$$

where J_{data} is the standard error (like MSE or Cross-Entropy), w is a single weight parameter, and λ is a small positive constant (hyperparameter).

- (a) Derive the Gradient Descent update rule for w . (i.e., write the equation for w_{new} in terms of w_{old} , the learning rate η , λ , and the data gradient $\frac{\partial J_{data}}{\partial w}$).
- (b) Rearrange your equation from part (a) to show that w_{old} is multiplied by a factor slightly less than 1 before the gradient update is applied.
- (c) Based on your answer in (b), explain in one sentence why L2 regularization is often technically referred to as "Weight Decay."

Let w_{old} be w & w_{new} be w' :

a.) differentiating for gradient :

$$\begin{aligned}\frac{\partial J_{total}}{\partial w} &= \frac{\partial J_{data}}{\partial w} + \frac{\partial}{\partial w} 2 \cdot \frac{\lambda}{2} w \\ &= \frac{\partial J_{data}}{\partial w} + \lambda w\end{aligned}$$

$$w' = w - \eta \frac{\partial J_{total}}{\partial w}$$

$$w' = w - \eta \left(\frac{\partial J_{data}}{\partial w} + \lambda w \right)$$

b.) Regrouping terms to rewrite (a) :

$$w' = (1 - \eta\lambda)w - \eta \frac{\partial J_{data}}{\partial w}$$

c.) Since we multiply the weight by $(1 - \eta\lambda)$ i.e. value < 1 at every step of the gradient descent step, the weight generally proceeds to continuously shrink or 'decay' towards 0. $\therefore$ L2 regulation is called 'weight decay'.

